CONFIDENTIAL TECHNICAL SPECIFICATION

Project Red String

Symmetric P2P Cross-Device Nexus & Reactive Telemetry Fabric

ARCH-V2.4
REVISION CERTIFIED

SYSTEM ARCHITECTURE	TRANSPORT PROTOCOL	FRAMING ENGINE	CRYPTOGRAPHIC TOKEN
Pure Dual-Node P2P	**WebRTC / mDNS LAN**	**RS-Binary 64KB Frame**	**Procedural Magic Circle**

1. Executive Summary & Design Philosophy

Red String is a high-performance, strictly symmetric communication infrastructure designed to bridge desktop workstations and mobile handhelds without intermediary cloud relays, third-party storage buckets, or account credential barriers. By establishing an autonomous, zero-knowledge peer-to-peer (P2P) link over direct local networks (or fault-tolerant STUN conduits), Red String guarantees sub-5ms operational latency, private direct-memory streaming, and fluid bidirectional asset coordination.

> ◆ **The Red String Core Axiom**
> A pair of linked nodes forms a sovereign, closed computational circuit. No assets touch centralized infrastructure. All state changes, live typing telemetries, and media drops live strictly within allocated volatile bounds negotiated via procedural visual cryptography.

2. Core Functional Pillars

The Platform

A finite, dual-tier volatile storage staging zone for photos, master audio streams, archives, and documents. Enforces strict soft caps with elastic buffer margins.

Fate Sync

Bidirectional interactive command and text exchange. Equipped with live clipboard synchronizers, bidirectional read/write permissions, and synchronized history.

Fate Watch

An asynchronous teleprompter and remote telemetry viewer. Renders transmitter keystrokes and structural adjustments on the watcher screen in sub-frame real time.

3. High-Level Architectural Topology

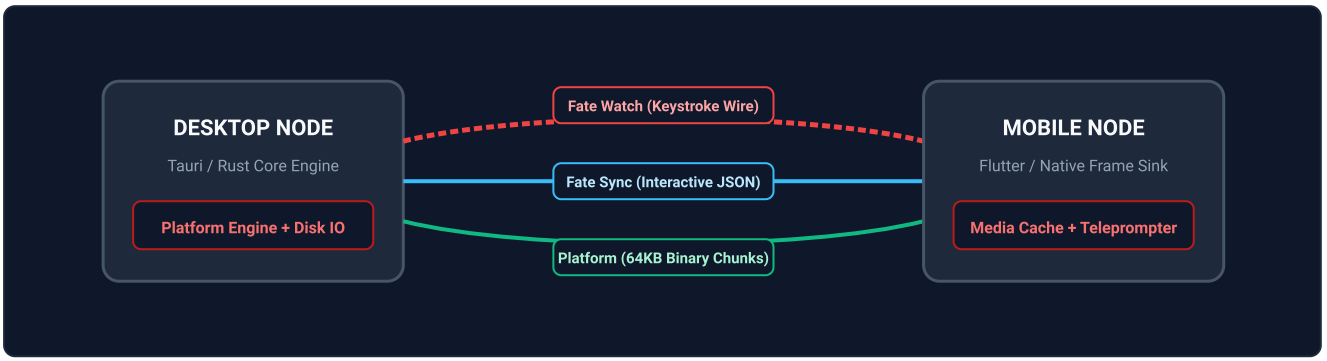


Figure 1.0: Symmetric Peer-to-Peer Bus detailing dual multiplexed transport channels.

4. The Magic Circle: Procedural Visual Cryptography

Conventional square barcodes (QR code standard ISO/IEC 18004) are inherently utilitarian and visually disconnected from narrative-driven interfaces. Red String introduces **The Magic Circle QR**: a hybrid visual encoding paradigm combining a high-density, fault-tolerant central matrix with procedurally synthesized concentric orbital runic tracks that scale dynamically in geometric complexity relative to the user's allocated Platform size.

Radial Layer Architecture

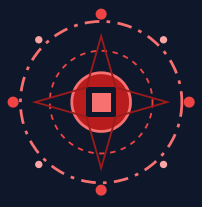
The token is composed of four concentric radii:

- **Core (0–35% R):** Micro-Aztec matrix with Reed-Solomon Level-H error correction holding symmetric cryptographic keys.
- **Mantle (35–65% R):** Runic parameter arcs encoding buffer size constraints and local socket endpoints.
- **Periphery (65–100% R):** Procedural orbital nodes whose angular frequency scales directly with memory allocation.

Complexity Scaling Mechanics

As the host expands Platform capacity, mathematical complexity increments:

- **512 MB Allocation:** Single ring, 4 geometric cardinal anchors, 32 polar glyph nodes.
- **2.0 GB Allocation:** Dual concentric counter-rotating rings, 8 octagonal vertices, 96 polar nodes.
- **10+ GB Allocation:** Triple orbital halo, complex non-Euclidean cross-hatching, 256 polar glyph nodes.



Dynamic Magic Circle Geometry Sample (2.0 GB Mode)

The mobile device uses camera hardware running an optimized thresholding pass to detect the four cardinal outer nodes ($r=44$), unwarps the polar coordinate matrix into a linear bitstream, and decrypts the P2P connection handshake payload in <120ms.

5. Dynamic Handshake Payload Specification

The encrypted string decoded from the Magic Circle translates into the following canonical JSON configuration schema before the WebRTC STUN/LAN discovery process triggers:

Field Key	Primitive Type	Example Value	Functional Role
`rs_ver`	`uint8`	`2`	Protocol compatibility validator.
`session_nonce`	`hex_string (64)`	`"a9c2...41bf"`	Single-use ephemeral ECDH handshake entropy.
`lan_discovery_port`	`uint16`	`49152`	Direct mDNS / UDP broadcast port for LAN.
`platform_cap_mb`	`uint32`	`2048`	Host-defined soft ceiling for volatile drop zone.
`flex_margin_pct`	`uint8`	`15`	Allowed buffer overrun percentage (Elasticity).

6. The Platform: Volatile Memory Limits & Elasticity

Unlike naive cloud drives, the **Red String Platform** operates with deterministic bounds. When configuring the connection, the host establishes a *Soft Cap*. However, to eliminate frustrating transfer failures when transferring large multi-file archives or camera dumps, Red String introduces a formal **Elastic Flex Zone**.

Zone 1: Nominal Range (≤ 100% Soft Cap)
Full read, write, and batch transfer permitted. Transfers stream into fast uncompressed memory structures or staging scratch space with real-time UI capacity feedback.

Zone 2: Elastic Margin (100% to 115%)
When transfers push occupancy into the elastic margin, the UI engages an amber warning. The buffer marks transient files for LRU (Least Recently Used) eviction upon disconnection.

7. Red String Binary Framing Engine (Channel 1)

To prevent memory exhaustion during massive 4K raw video or multi-gigabyte document transfers, files bypass JSON parsing entirely and stream across dedicated WebRTC DataChannels in atomic **64 KB binary frames** with a packed 24-byte hardware-aligned header.

Byte Offset	Field	Type	Structural Purpose
`0x00 - 0x01`	Magic Prefix	`uint16_t`	Constant sentinel `0x5253` ('RS' for Red String framing sync).
`0x02`	Frame Version	`uint8_t`	Protocol framing revision identifier (`0x01`).
`0x03`	Flags	`uint8_t`	Bit 0: Terminal Chunk (EOF); Bit 1: Flex Zone Asset; Bit 2: Compressed.
`0x04 - 0x07`	File ID	`uint32_t`	Unique transaction manifest reference identifier.
`0x08 - 0x0B`	Chunk Index	`uint32_t`	Monotonically increasing sequence offset (`0, 1, 2...`).
`0x0C - 0x0F`	Payload Length	`uint32_t`	Bytes enclosed in raw payload (Nominal: 65,536; variable on tail chunk).
`0x10 - 0x17`	CRC-64 Checksum	`uint64_t`	ECMA-182 polynomial checksum for frame-level corruption isolation.
`0x18 - End`	Raw Payload	`byte[]`	Unmodified sliced binary segment written directly to destination pipe.

◆ **Zero-Copy Architecture Note**
By aligning the header to exact 8-byte boundaries and fixing offsets, the native Rust runtime on Desktop and mobile C++ JNI/NDK layers perform zero-copy deserialization directly into the operating system's kernel disk write buffers.

8. Fate Sync vs. Fate Watch Telemetry Engine

The text streaming plane provides two radically distinct modes designed for active collaboration versus detached remote viewing:

Fate Sync: Bidirectional Mesh

- Both nodes hold mutual read/write permissions.
- Includes an automated bidirectional clipboard synchronizer with regex filtering (prevents password spills).
- Packets carry delivery acknowledgments, typing presence flags, and threaded context anchors.

Fate Watch: Ghost Teleprompter

- Transmitter acts as broadcast source; Receiver is locked into read-only observation mode.
- Instead of full message submits, transmits sub-character operational transformation deltas (`insert` , `delete` , `cursor_pos`).
- Mobile display operates as a real-time distraction-free typewriter head with glowing crimson cursor rendering.

9. Cryptographic & Security Baseline

Vector	Implementation Strategy	Security Guarantee
In-Flight Encryption	DTLS 1.3 / SRTP over WebRTC channels.	Zero plaintext eavesdropping on untrusted Wi-Fi.
Key Ephemerality	X25519 Curve exchange per Magic Circle.	Perfect Forward Secrecy; past keys cannot decrypt future links.
Memory Sanitization	Zero-fill overwrite on session disconnection.	Prevents forensic memory dump recovery of Platform drops.

10. Implementation & Delivery Roadmap

Phase I: Core Weave

Weeks 1–4

- Tauri + Flutter framework setup
- WebRTC DataChannel link
- Basic 64KB binary framer

Phase II: Visual Rite

Weeks 5–7

- Magic Circle SVG engine
- Mobile OpenCV polar decoder
- Adaptive size scaling logic

Phase III: Fate Telemetry

Weeks 8–10

- Fate Watch live character diffs
- Fate Sync clipboard bridges
- Flex limit LRU management

[Project Red String Engineering Directorate • Ready for Core Implementation]